

**T.C.**  
**KTO KARATAY ÜNİVERSİTESİ**  
**MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ**  
**Bilgisayar Mühendisliği Bölümü**



**2025-2026 BAHAR DÖNEMİ**

**HABERLEŞME MÜHENDİSLİĞİ**  
**FİNAL RAPORU**

**ESP-NOW İLE SÜRÜ AĞI TEKNOLOJİLERİ**

**HAZIRLAYAN:**  
**221450015 – Fatma ÇELİK**

**24.05.2026**

## ÖZET

Bu çalışmada, Haberleşme Mühendisliği dersi kapsamında ESP-NOW protokolü kullanılarak üç düğümlü bir kablosuz haberleşme ve sürü ağı MVP prototipi geliştirilmiştir. Proje, ara sınav raporunda teorik olarak incelenen ESP-NOW teknolojisinin final aşamasında uygulamalı bir sisteme dönüştürülmesini amaçlamaktadır. Ara raporda ESP-NOW'un bağlantısız yapısı, düşük gecikmeli iletişim sağlaması, 2.4 GHz bandında çalışması ve çok düğümlü sistemlerde kullanılabilirliği ele alınmıştır.

Final prototipinde üç adet ESP32-WROOM-32D kart kullanılmıştır. Kartlardan biri Master + Node 1, diğerleri ise Node 2 ve Node 3 olarak yapılandırılmıştır. Her düğüme dijital eşik çıkışlı mikrofon modülü ve kırmızı/yeşil LED göstergeleri bağlanmıştır. Mikrofon modülünün ses algıladığında LOW sinyali verdiği test edilmiş ve sistem bu aktif-LOW mantığına göre düzenlenmiştir. Ses algılayan node, ESP-NOW üzerinden master düğüme olay paketi göndermekte; master ise ilk gelen olay paketini “sese en yakın/ilk algılayan node” olarak kabul edip sonucu tüm node'lara broadcast olarak göndermektedir. Sonuç olarak kazanan node kırmızı LED yakmakta, diğer node'lar yeşil LED durumunda kalmaktadır.

Bu proje, hassas TDOA tabanlı bir konum belirleme sistemi değil; ESP-NOW tabanlı sürü haberleşmesini, node kimliğiyle paket ayrımını, merkezi karar mekanizmasını ve kablosuz geri bildirim sürecini gösteren bir MVP prototipidir.

## 1. İindekiler

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>ÖZET.....</b>                                                               | <b>2</b>  |
| <b>TABLolar LİSTESİ.....</b>                                                   | <b>4</b>  |
| <b>ŞEKİLLER LİSTESİ.....</b>                                                   | <b>4</b>  |
| <b>1. GİRİŞ .....</b>                                                          | <b>5</b>  |
| <b>2. Projenin Amacı .....</b>                                                 | <b>5</b>  |
| <b>3. ESP-NOW Teknolojisi.....</b>                                             | <b>6</b>  |
| <b>4. Sürü Ağı Teknolojisi.....</b>                                            | <b>6</b>  |
| <b>5. Donanım Seçimi ve ESP32-S3 Yerine ESP32-WROOM-32D Kullanılması .....</b> | <b>7</b>  |
| <b>6. Materyal ve Metot.....</b>                                               | <b>9</b>  |
| <b>7. Bağlantılar.....</b>                                                     | <b>10</b> |
| 7.1. Mikrofon Modülü Bağlantısı.....                                           | 10        |
| 7.2. LED Bağlantısı .....                                                      | 10        |
| <b>8. Sistem Mimarisi .....</b>                                                | <b>11</b> |
| <b>9. Program Akışı.....</b>                                                   | <b>12</b> |
| <b>10. Kodların Açıklanması.....</b>                                           | <b>12</b> |
| 10.1. Master + Node 1 Kodu .....                                               | 12        |
| 10.2. Node 2 ve Node 3 Kodu .....                                              | 13        |
| <b>11. Karşılaşılan Sorunlar ve Çözümler .....</b>                             | <b>14</b> |
| 11.1. HC-SR04 Sensörlü İlk MVP Aşaması .....                                   | 14        |
| 11.2. Mikrofon Modülünün Analog Genlik Vermemesi.....                          | 14        |
| 11.3. Aktif LOW Mikrofon Mantığı .....                                         | 14        |
| <b>12. Test Süreci ve Bulgular.....</b>                                        | <b>15</b> |
| <b>13. Echo Swarm ile İlişkilendirme .....</b>                                 | <b>16</b> |
| <b>14. Sonuç .....</b>                                                         | <b>17</b> |
| <b>KAYNAKÇA.....</b>                                                           | <b>17</b> |
| <b>EKLER.....</b>                                                              | <b>18</b> |

## TABLÖLAR LİSTESİ

|                                             |    |
|---------------------------------------------|----|
| Tablo 1: Kullanılan Malzemeler.....         | 9  |
| Tablo 2: Mikrofon Modülü Bağlantıları ..... | 10 |
| Tablo 3: LED Bağlantıları .....             | 10 |

## ŞEKİLLER LİSTESİ

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| Şekil 1: Sürü Ağı Teknolojileri.....                                    | 7  |
| Şekil 2: ESP32-WROOM-32D Geliştirme Kartı.....                          | 8  |
| Şekil 3: HC-SR04 sensörlü ilk MVP prototipi .....                       | 9  |
| Şekil 4: Mikrofon modüllü final prototip .....                          | 9  |
| Şekil 5: ESP-NOW Sistem Mimarisi .....                                  | 11 |
| Şekil 6: HC-SR04 sensörlü ESP-NOW ilk MVP prototipi.....                | 14 |
| Şekil 7: Bir node'un kırmızı, diğerlerinin yeşil yandığı test anı ..... | 15 |
| Şekil 8: Seri monitörde event/result mesajlarının gözlemlenmesi .....   | 16 |
| Şekil 9: Echo-Swarm (Temsili).....                                      | 16 |

## 1. GİRİŞ

Kablosuz haberleşme sistemleri; sensör ağları, gömülü sistemler, nesnelerin interneti ve robotik uygulamalar için temel bir altyapı oluşturmaktadır. Geleneksel Wi-Fi sistemlerinde cihazlar çoğunlukla bir modem, yönlendirici veya erişim noktası üzerinden haberleşir. Ancak bazı uygulamalarda dış bir altyapıya bağlı kalmadan cihazların doğrudan birbirleriyle haberleşmesi gerekir. Özellikle çok düğümlü sensör ağları ve sürü robotları gibi yapılarda düşük gecikme, hızlı veri aktarımı ve esnek ağ mimarisi önemli hâle gelmektedir [1].

ESP-NOW, Espressif tarafından geliştirilen bağlantısız bir Wi-Fi haberleşme protokolüdür. Resmî dokümantasyonda ESP-NOW'un uygulama verisini vendor-specific action frame yapısı içinde taşıdığı ve bağlantı kurmadan Wi-Fi cihazları arasında veri iletebildiği belirtilmektedir [2]. Bu yönüyle ESP-NOW, küçük veri paketlerinin hızlı biçimde aktarılması gereken sensör ve kontrol uygulamaları için uygun bir protokoldür.

Bu çalışmada ESP-NOW teknolojisi, sürü ağı mantığıyla birlikte ele alınmış ve üç adet ESP32 kart kullanılarak ses algılama tabanlı bir MVP prototip oluşturulmuştur. Projede ana amaç, birden fazla düğümün aynı ağ içinde algılama yapması, algılanan olayın master düğüme iletilmesi ve master düğümün sonucu tekrar tüm düğümlere iletilmesidir.

## 2. Projenin Amacı

Bu projenin temel amacı, ESP-NOW protokolünün çok düğümlü haberleşme sistemlerinde nasıl kullanılabileceğini uygulamalı olarak göstermektir. Ara sınav raporunda ESP-NOW teknolojisi; kablosuz sensör ağları, paket yapısı, frekans bandı ve sürü ağı yapıları açısından teorik olarak incelenmiştir. Final aşamasında ise bu teorik altyapı çalışan bir prototipe dönüştürülmüştür.

Projenin amaçları şunlardır:

- ESP32 kartlar arasında modem veya router kullanmadan haberleşme sağlamak.
- ESP-NOW broadcast yapısını kullanarak node'lar arasında veri aktarımı yapmak.
- Her node'un kendi mikrofon modülüyle ses algılamasını sağlamak.
- Ses algılayan node'un master'a olay paketi göndermesini sağlamak.
- Master düğümde ilk gelen olay paketini değerlendirmek.
- Sonucu tüm node'lara geri göndermek.
- Kazanan node'u kırmızı LED, diğer node'ları yeşil LED ile göstermek.
- Sürü ağı haberleşmesini görsel ve anlaşılır bir prototip üzerinden sunmak.

Bu amaç doğrultusunda proje iki aşamada ilerlemiştir. İlk aşamada HC-SR04 ultrasonik mesafe sensörüyle engel algılama tabanlı bir ESP-NOW demosu hazırlanmıştır. Bu sistemde master düğüm engel durumunu algılayıp Node 2 ve Node 3'e LED komutu göndermiştir.

Sunumda da bu MVP, master düğümün HC-SR04 ile mesafe/engel algıladığı ve ESP-NOW broadcast ile diğer node'lara komut gönderdiği yapı olarak anlatılmıştır. İkinci aşamada ise proje mikrofon modüllü akustik algılama sistemine dönüştürülmüştür.

### 3. ESP-NOW Teknolojisi

ESP-NOW, ESP32/ESP8266 tabanlı cihazların modem veya erişim noktası kullanmadan doğrudan haberleşmesini sağlayan kablosuz haberleşme protokolüdür. ESP-NOW, klasik Wi-Fi bağlantısında olduğu gibi uzun bağlantı kurulum süreçlerine ihtiyaç duymaz. Bu nedenle kısa veri paketlerinin hızlı iletimi için uygundur [2].

ESP-NOW'un temel özellikleri şu şekilde özetlenebilir:

- 2.4 GHz frekans bandında çalışır.
- Bağlantısız Wi-Fi haberleşmesi sağlar.
- Harici modem veya erişim noktası gerektirmez.
- Broadcast ve unicast haberleşmeyi destekler.
- Küçük veri paketlerinin iletimi için uygundur.
- Düşük gecikmeli sensör ve kontrol uygulamalarında kullanılabilir.
- CCMP tabanlı güvenlik yapısı destekler [2].

Espressif dokümantasyonunda ESP-NOW'un akıllı aydınlatma, uzaktan kontrol ve sensör uygulamaları gibi alanlarda kullanılabildiği belirtilmektedir [2]. Random Nerd Tutorials gibi uygulama kaynaklarında da ESP-NOW'un ESP32 kartlar arasında Arduino IDE ile kısa paket iletimi için yaygın olarak kullanıldığı görülmektedir [7].

Bu projede ESP-NOW'un broadcast özelliği kullanılmıştır. Broadcast için aşağıdaki genel yayın adresi tanımlanmıştır:

```
uint8_t broadcastAddress[] = {  
  0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF  
};
```

Bu yapı sayesinde node'lar belirli bir MAC adresi kullanmadan aynı anda tüm düğümlere veri gönderebilmiştir. Node ayrımı ise paket içinde taşınan nodeId değeri ile yapılmıştır.

### 4. Sürü Ağı Teknolojisi

Sürü ağı teknolojisi, birden fazla küçük düğümün birlikte çalışması prensibine dayanır. Her düğüm veri algılayabilir, veri gönderebilir veya sistemin genel davranışına katkı sağlayabilir. Ara sınav raporunda da ESP-NOW ile kurulan sürü ağlarında düğümlerin

merkezi birime ihtiyaç duymadan doğrudan haberleşebileceği ve sistemin yeni cihazlar eklenerek genişletilebileceği açıklanmıştır.



**Şekil 1: Sürü Ağı Teknolojileri**

Bu projede sürü ağı mantığı üç düğümlü bir yapı ile temsil edilmiştir. Node 1 hem algılayıcı hem de master karar verici düğüm olarak çalışmaktadır. Node 2 ve Node 3 ise bağımsız algılayıcı düğümlerdir. Her node kendi mikrofon modülüyle ses olayını algılamakta ve ESP-NOW üzerinden master'a veri göndermektedir.

Bu yapı, sürü sistemleri açısından şu süreçleri göstermektedir:

- Dağıtık algılama
- Kablosuz veri aktarımı
- Node kimliğiyle veri ayırımı
- Merkezi karar verme
- Sonucun tüm ağa yayınlanması
- LED ile görsel geri bildirim

## **5. Donanım Seçimi ve ESP32-S3 Yerine ESP32-WROOM-32D Kullanılması**

Ara sınav raporunda ESP32-S3 kartı, yüksek işlem gücü, DSP/yapay zekâ uygulamalarına uygunluğu ve gelişmiş donanım özellikleri nedeniyle önerilen platform olarak değerlendirilmiştir. Raporda ESP32-S3'ün özellikle sinyal işleme ve yapay zekâ uygulamaları için avantajlı olduğu belirtilmiştir.

Ancak final prototipinde üç adet **ESP32-WROOM-32D** kart kullanılmıştır. Bu değişikliğin temel nedeni, final aşamasında gerçekleştirilen MVP sistemin doğrudan TinyML veya yoğun DSP işlemi gerektirmemesidir. Son sistemde mikrofon modüllerinden gerçek analog ses verisi işlenmemiş, bunun yerine dijital eşik çıkışı kullanılmıştır. Yani sistemde ses verisinin sınıflandırılması, yapay zekâ tabanlı analiz veya karmaşık sinyal işleme yapılmamıştır. Bu nedenle ESP32-S3'ün yüksek işlem gücüne ihtiyaç duyulmamıştır.



**Şekil 2: ESP32-WROOM-32D Geliştirme Kartı**

ESP32-WROOM-32D kartının tercih edilme gerekçeleri şunlardır:

- Elde üç adet aynı model ESP32-WROOM-32D kart bulunması.
- ESP-NOW protokolünü desteklemesi.
- Dijital mikrofon çıkışı ve LED kontrolü için yeterli GPIO pinine sahip olması.
- Arduino IDE ile kolay programlanabilmesi.
- MVP seviyesinde broadcast haberleşme, event paketi ve result paketi iletimi için yeterli olması.
- Üç düğümlü sistemin aynı kart modeliyle daha kararlı ve tutarlı kurulabilmesi.

Bu nedenle ESP32-S3, projenin ileri aşamaları için daha uygun bir platform olarak değerlendirilirken; finaldeki MVP prototip için ESP32-WROOM-32D kartları yeterli görülmüştür. Gelecekte analog/I2S mikrofon, TinyML tabanlı insan sesi algılama veya akustik konum tahmini gibi işlemler eklendiğinde ESP32-S3 kartına geçilmesi daha anlamlı olacaktır.

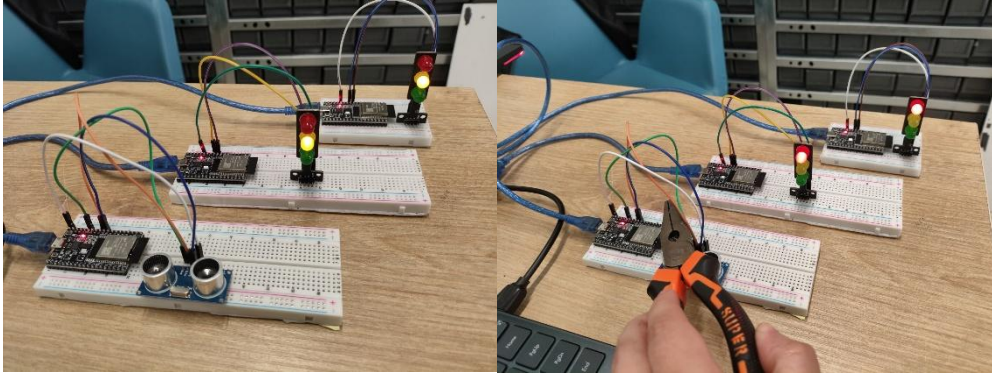


## 6. Materyal ve Metot

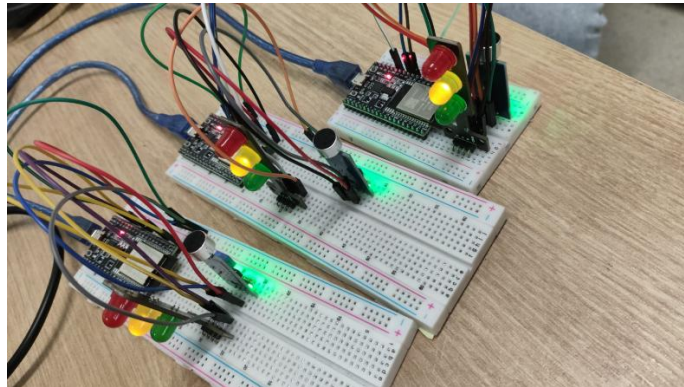
Projede kullanılan temel donanım bileşenleri aşağıdaki gibidir:

**Tablo 1: Kullanılan Malzemeler**

| Bileşen                           | Adet    | Görev                                  |
|-----------------------------------|---------|----------------------------------------|
| ESP32-WROOM-32D                   | 3       | Master ve node kontrolcüler            |
| Mikrofon / ses sensörü modülü     | 3       | Ses olayının algılanması               |
| HC-SR04 ultrasonik mesafe sensörü | 1       | İlk MVP aşamasında engel algılama      |
| Kırmızı LED                       | 3       | Kazanan node'u göstermek               |
| Yeşil LED                         | 3       | Bekleme/normal node durumunu göstermek |
| 220 $\Omega$ direnç               | 6       | LED akım sınırlama                     |
| Jumper kablolar                   | Yeterli | Devre bağlantıları                     |
| Breadboard                        | Yeterli | Prototip kurulumu                      |



**Şekil 3: HC-SR04 sensörlü ilk MVP prototipi**



**Şekil 4: Mikrofon modüllü final prototip**

## 7. Bağlantılar

### 7.1. Mikrofon Modülü Bağlantısı

Mikrofon modülü dijital eşik çıkışı ile kullanılmıştır. Testler sonucunda modülün ses algılandığında LOW verdiği görülmüştür. Bu nedenle kodda aktif ses durumu şu şekilde tanımlanmıştır:

```
#define SOUND_ACTIVE_STATE LOW
```

Her üç node için mikrofon bağlantısı:

**Tablo 2: Mikrofon Modülü Bağlantıları**

| Mikrofon Modülü | ESP32-WROOM-32D |
|-----------------|-----------------|
| VCC             | 3.3V            |
| GND             | GND             |
| OUT             | GPIO32          |

### 7.2.LED Bağlantısı

Her node’da kırmızı ve yeşil LED kullanılmıştır:

**Tablo 3: LED Bağlantıları**

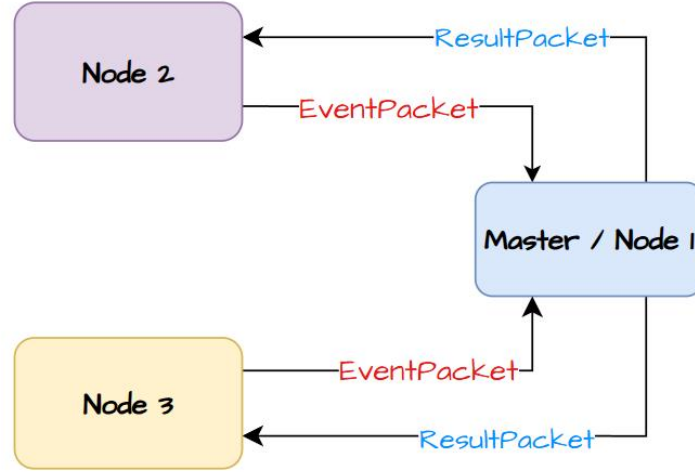
| LED         | ESP32 Pini |
|-------------|------------|
| Kırmızı LED | GPIO26     |
| Yeşil LED   | GPIO27     |
| LED GND     | GND        |

Trafik lambası modülü kullanıldığında pinler şu şekilde bağlanmıştır:

```
R → GPIO26  
G → GPIO27  
Y → Boş  
GND → GND
```

## 8. Sistem Mimarisi

Sistem üç düğümlü master-node yapısına sahiptir:



Şekil 5: ESP-NOW Sistem Mimarisi

Master/Node 1 hem kendi mikrofonundan ses algılayabilmekte hem de Node 2 ve Node 3'ten gelen event paketlerini dinlemektedir. Node 2 ve Node 3 yalnızca kendi mikrofonlarını dinlemekte, ses algılandığında master'a event paketi göndermekte ve master'dan gelen result paketine göre LED durumunu güncellemektedir.

Sistemde iki temel veri paketi kullanılmıştır:

```
typedef struct __attribute__((packed)) {  
    uint8_t msgType;  
    uint8_t nodeId;  
    uint32_t eventId;  
    uint32_t eventTime;  
} EventPacket;
```

```
typedef struct __attribute__((packed)) {  
    uint8_t msgType;  
    uint8_t winnerId;  
    uint32_t decisionId;  
} ResultPacket;
```

**EventPacket**, ses algılayan node tarafından master'a gönderilmektedir. **ResultPacket** ise master tarafından kazanan node bilgisini tüm node'lara iletmek için kullanılmaktadır.

## 9. Program Akışı

Program çalıştırıldığında tüm ESP32 kartlar WIFI\_STA moduna alınır ve ESP-NOW başlatılır. Daha sonra broadcast peer eklenir ve sistem dinleme moduna geçer.

Programın genel akışı şu şekildedir:

1. ESP32 kart başlatılır.
2. ESP-NOW başlatılır.
3. Broadcast peer eklenir.
4. Yeşil LED yakılarak bekleme durumu gösterilir.
5. Mikrofon OUT pini okunur.
6. OUT = LOW ise ses algılandı kabul edilir.
7. Ses algılayan node master'a EventPacket gönderir.
8. Master ilk gelen event paketini winnerId olarak kaydeder.
9. Master kısa karar penceresi sonunda ResultPacket yayınlar.
10. winnerId kendi node ID'siyle aynı olan node kırmızı LED yakar.
11. Diğer node'lar yeşil LED durumunda kalır.
12. 3 saniye sonra sistem tekrar dinleme moduna döner.

Bu akış, draw.io ile hazırlanan akış şemasında “Başlat → ESP-NOW kurulumu → Mikrofon OUT oku → OUT = LOW mu? → EventPacket gönder → Master karar verir → ResultPacket yayınla → LED durumunu güncelle” şeklinde gösterilebilir.

## 10. Kodların Açıklanması

Raporun ana metninde kodların tamamını uzun uzun anlatmak yerine, görevlerini açıklayıp tam kodları Ekler bölümüne koymak daha doğru olur.

### 10.1. Master + Node 1 Kodu

Master kodu iki görevi aynı anda yürütmektedir. Birincisi Node 1 olarak kendi mikrofonunu dinlemek, ikincisi ise Node 2 ve Node 3'ten gelen event paketlerini alıp sonucu belirlemektir.

Master kodunda kullanılan temel bölümler:

- **esp\_now\_init()** ile ESP-NOW başlatılır.
- Broadcast adresi **peer** olarak eklenir.

- **OnDataRecv()** fonksiyonu ile gelen event paketleri dinlenir.
- **registerEvent()** fonksiyonu ilk gelen node'u kaydeder.
- **sendResult()** fonksiyonu kazanan node bilgisini tüm ağa yayınlar.
- **resetRound()** fonksiyonu 3 saniye sonunda sistemi yeni olaya hazırlar.

Master kodunda önemli tanımlar şunlardır:

```
#define NODE_ID 1
#define MIC_OUT_PIN 32
#define RED_LED_PIN 26
#define GREEN_LED_PIN 27
#define SOUND_ACTIVE_STATE LOW
#define DECISION_WINDOW_MS 400
#define RESULT_HOLD_MS 3000
```

## 10.2. Node 2 ve Node 3 Kodu

Node 2 ve Node 3 aynı kod yapısını kullanmaktadır. Sadece NODE\_ID değeri değiştirilmiştir.

Node 2 için:

```
#define NODE_ID 2
```

Node 3 için:

```
#define NODE_ID 3
```

Node kodlarının temel görevleri:

- Mikrofon OUT pinini okumak.
- Ses algılanırsa master'a EventPacket göndermek.
- Master'dan gelen ResultPacket paketini dinlemek.
- winnerId kendi ID'sine eşitse kırmızı LED yakmak.
- winnerId farklıysa yeşil LED yakmak.

Node kodunda kritik karar satırı:

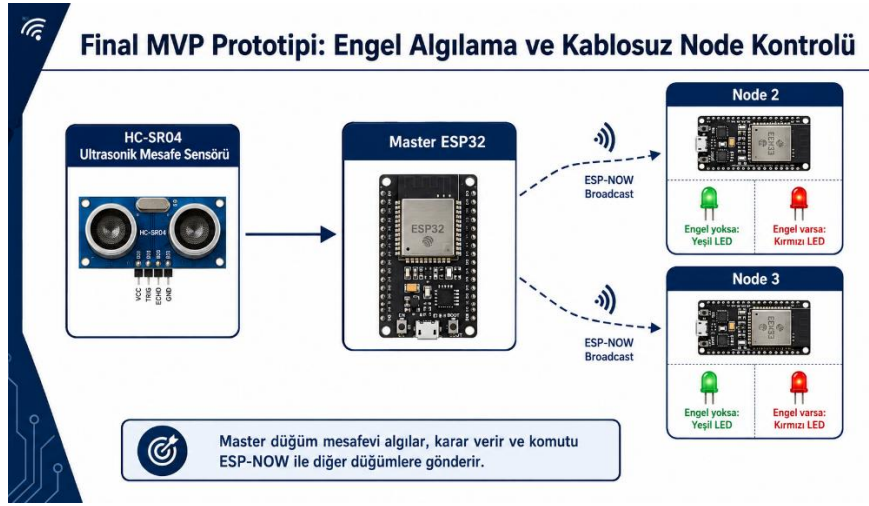
```
if (micState == SOUND_ACTIVE_STATE &&
    millis() - lastSendMs > SEND_COOLDOWN_MS) {
    sendEvent();
}
```

Bu satır, mikrofon LOW olduğunda ve soğuma süresi dolduğunda event paketinin gönderilmesini sağlar.

## 11. Karşılaşılan Sorunlar ve Çözümler

### 11.1. HC-SR04 Sensörlü İlk MVP Aşaması

İlk aşamada ESP-NOW haberleşmesini daha kararlı ve sınıfta kolay gösterilebilir hâle getirmek için HC-SR04 sensörlü bir sistem kurulmuştur. Bu yapıda master düğüm mesafe bilgisini okuyarak engel var/yok kararını üretmiş ve Node 2 ile Node 3'e LED komutu göndermiştir. Bu aşama, ESP-NOW broadcast mantığını doğrulamak için kullanılmıştır. Final sunumunda da master düğümün HC-SR04 ile mesafe/engel algıladığı ve node'lara ESP-NOW komutu gönderdiği anlatılmıştır.



Şekil 6: HC-SR04 sensörlü ESP-NOW ilk MVP prototipi

### 11.2. Mikrofon Modülünün Analog Genlik Vermemesi

İlk hedef, mikrofon modülünden analog ses genliği okuyarak en yüksek genliğe sahip node'u sese en yakın node olarak belirlemektir. Ancak testlerde modülün analog çıkışının kararlı bir ses genliği vermediği görülmüştür. Potansiyometre çevrildiğinde çıkış değerlerinin 0 ve 4095 gibi uç değerlere kaydığı, gerçek ses genliği ölçümü için uygun olmadığı anlaşılmıştır.

Bu nedenle sistem analog genlik karşılaştırmasından dijital eşik tabanlı algılama mantığına dönüştürülmüştür. Son sistemde mikrofon modülünün OUT pini kullanılmış ve ses algılandığında LOW olduğu belirlenmiştir.

### 11.3. Aktif LOW Mikrofon Mantığı

Mikrofon test koduyla şu yapı doğrulanmıştır:

```
if (sesDurumu == LOW) {  
  Serial.println("Ses Algılandı!");  
}
```

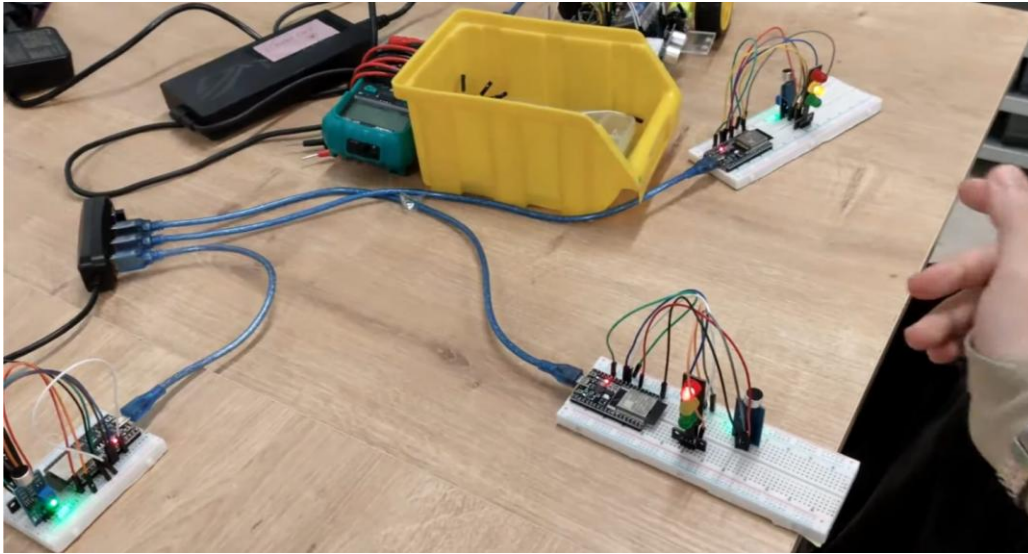
Bu test sonucunda modülün aktif LOW çalıştığı anlaşılmıştır. Ana kodda da bu durum SOUND\_ACTIVE\_STATE LOW olarak tanımlanmıştır.

## 12. Test Süreci ve Bulgular

Testler aşamalı olarak gerçekleştirilmiştir. İlk olarak master düğüm tek başına denenmiştir. Master mikrofona yakın ses çıkarıldığında seri ekranda “NODE 1 SES ALGILADI” mesajı gözlemlenmiş ve kırmızı LED’in yandığı doğrulanmıştır.

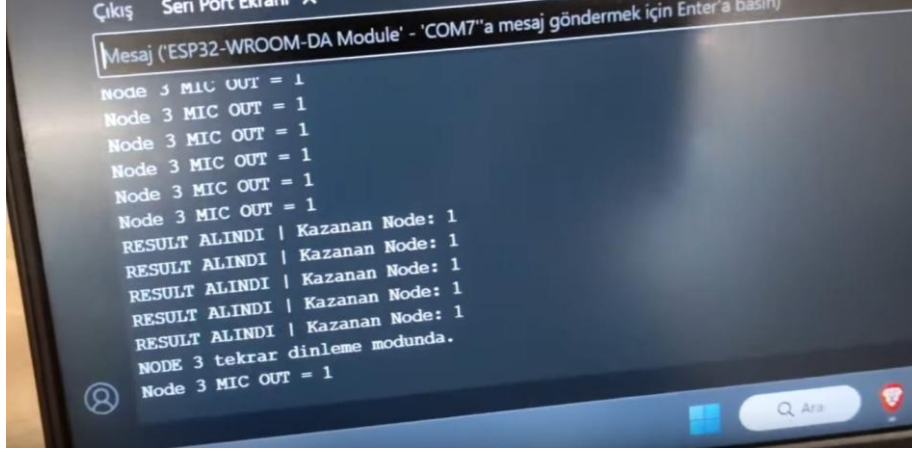
Daha sonra Node 2 sisteme eklenmiştir. Node 2 mikrofona ses algıladığında “NODE 2 SES ALGILADI” ve “EVENT GONDERILDI” mesajları alınmıştır. Master düğümün bu event paketini almasıyla kazanan node belirlenmiş ve sonuç paketi yayınlanmıştır.

Son aşamada Node 3 de sisteme eklenmiştir. Üç düğüm birlikte çalıştırıldığında, sesi ilk algılayan node’un kırmızı LED yaktığı, diğer düğümlerin yeşil LED durumunda kaldığı gözlemlenmiştir.



Şekil 7: Bir node’un kırmızı, diğerlerinin yeşil yandığı test anı





Şekil 8: Seri monitörde event/result mesajlarının gözlemlenmesi

### 13. Echo Swarm ile İlişkilendirme

Bu çalışma, daha ileri bir proje fikri olan Echo Swarm için temel haberleşme altyapısı olarak değerlendirilebilir. Echo Swarm; enkaz ve arama-kurtarma senaryolarında birden fazla küçük robotun çevreden ses verisi topladığı, birbirleriyle haberleştiği ve ortak bir algılama ağı oluşturduğu bir sistem olarak düşünülmektedir. Final sunumunda da Echo Swarm'ın enkaz altında çok düğümlü akustik algılama ve haberleşme sistemi olarak ele alındığı görülmektedir.

Bu final projesinde geliştirilen ESP-NOW tabanlı MVP, Echo Swarm'ın haberleşme mantığının basitleştirilmiş bir prototipi olarak düşünülebilir. Mevcut sistemde gerçek insan sesi sınıflandırması veya hassas TDOA konum kestirimi yapılmamaktadır. Ancak node'ların ses algılaması, master'a veri göndermesi ve master'ın sonucu tüm ağa yayınlaması, Echo Swarm gibi sistemler için temel bir haberleşme yaklaşımını göstermektedir.



Şekil 9: Echo-Swarm (Temsili)



## 14. Sonuç

Bu projede ESP-NOW protokolü kullanılarak üç düğümlü bir sürü ağı haberleşme prototipi geliştirilmiştir. Proje, ara sınav raporunda teorik olarak incelenen ESP-NOW teknolojisinin final aşamasında çalışan bir uygulamaya dönüştürülmesini sağlamıştır. ESP-NOW'un altyapı gerektirmeyen yapısı, broadcast haberleşme desteği ve kısa paketlerle düşük gecikmeli veri aktarımına uygun olması, bu proje için temel avantajlar olarak değerlendirilmiştir [2], [7].

Final sisteminde üç adet ESP32-WROOM-32D kart kullanılmıştır. Ara raporda ESP32-S3 önerilmiş olsa da, final MVP sisteminde yoğun yapay zekâ veya DSP işlemi yapılmadığı için ESP32-WROOM-32D kartlarının yeterli olduğu görülmüştür. Bu durum, mühendislik tasarım sürecinde gereksinime göre donanım seçimi yapılmasının önemini göstermektedir.

Sistem başlangıçta HC-SR04 sensörlü engel algılama prototipiyle doğrulanmış, daha sonra mikrofon modüllü ses algılama yapısına dönüştürülmüştür. Mikrofon modüllerinin analog genlik yerine dijital eşik çıkışı verdiği anlaşıldığından, sistem “ilk tetiklenen node” mantığıyla çalışacak şekilde düzenlenmiştir. Son durumda sesi ilk algılayan node master tarafından kazanan olarak belirlenmekte ve kırmızı LED ile gösterilmektedir.

Gelecekte sistem şu yönlerde geliştirilebilir:

- ESP32-S3 veya daha güçlü bir kart ile TinyML tabanlı insan sesi algılama eklenebilir.
- Analog veya I2S mikrofon kullanılarak gerçek ses genliği ölçülebilir.
- TDOA tabanlı akustik konum kestirimi yapılabilir.
- Broadcast yerine MAC adresi tabanlı unicast haberleşme eklenebilir.
- Daha fazla node ile ağ genişletilebilir.
- Mobil veya bilgisayar arayüzüyle node durumları izlenebilir.
- Echo Swarm benzeri robotik platformlara entegre edilebilir.

## KAYNAKÇA

[1] I. F. Akyildiz, W. Su, Y. Sankarasubramaniam ve E. Cayirci, “Wireless Sensor Networks: A Survey,” *Computer Networks*, 2002.

[2] Espressif Systems, “ESP-NOW - ESP-IDF Programming Guide.” Erişim: [https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp\\_now.html](https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html)

[3] Espressif Systems, “ESP-NOW User Guide.” Erişim: [https://github.com/espressif/esp-now/blob/master/User\\_Guide.md](https://github.com/espressif/esp-now/blob/master/User_Guide.md)

- [4] H. Karl ve A. Willig, *Protocols and Architectures for Wireless Sensor Networks*, Wiley, 2005.
- [5] T. S. Rappaport, *Wireless Communications: Principles and Practice*, Prentice Hall, 2002.
- [6] Espressif Systems, “ESP-NOW Wireless Communication Protocol.” Erişim: <https://www.espressif.com/en/solutions/low-power-solutions/esp-now>
- [7] Random Nerd Tutorials, “Getting Started with ESP-NOW ESP32 with Arduino IDE.” Erişim: <https://randomnerdtutorials.com/esp-now-esp32-arduino-ide/>
- [8] Fatma Çelik, “ESP-NOW ile Sürü Ağı Teknolojileri Ara Sınav Raporu,” KTO Karatay Üniversitesi, Haberleşme Mühendisliği, 2026.
- [9] Fatma Çelik, “ESP-NOW ile Sürü Ağı Teknolojileri Final Sunumu,” KTO Karatay Üniversitesi, Haberleşme Mühendisliği, 2026.

## EKLER

### EK-A: Master + Node 1 Mikrofonlu ESP-NOW Kodu

```
#include <esp_now.h>
#include <WiFi.h>
#include <esp_wifi.h>

#if __has_include(<esp_arduino_version.h>)
  #include <esp_arduino_version.h>
#endif

#define NODE_ID 1
#define ESPNOW_CHANNEL 1

#define MIC_OUT_PIN 32

#define RED_LED_PIN 26
#define GREEN_LED_PIN 27

#define MSG_EVENT 1
#define MSG_RESULT 2

// Kullanılan mikrofon modülü ses algıladığında OUT pini LOW seviyesine düşmektedir.
#define SOUND_ACTIVE_STATE LOW

#define DECISION_WINDOW_MS 400
#define RESULT_HOLD_MS 3000
#define LOCAL_COOLDOWN_MS 1200
```

```

uint8_t broadcastAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

typedef struct __attribute__((packed)) {
    uint8_t msgType;
    uint8_t nodeId;
    uint32_t eventId;
    uint32_t eventTime;
} EventPacket;

typedef struct __attribute__((packed)) {
    uint8_t msgType;
    uint8_t winnerId;
    uint32_t decisionId;
} ResultPacket;

bool roundActive = false;
bool resultSent = false;

uint32_t roundStartMs = 0;
uint32_t resultStartMs = 0;
uint32_t lastLocalEventMs = 0;

uint32_t localEventId = 0;
uint32_t decisionId = 0;

uint8_t winnerId = 0;

bool seen[4] = {false, false, false, false};

void setGreen() {
    digitalWrite(GREEN_LED_PIN, HIGH);
    digitalWrite(RED_LED_PIN, LOW);
}

void setRed() {
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, HIGH);
}

void alloff() {
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, LOW);
}

void registerEvent(uint8_t id) {
    if (id < 1 || id > 3) return;
    if (resultSent) return;

```

```

if (!roundActive) {
    roundActive = true;
    roundStartMs = millis();

    // MVP mantığı:
    // İlk ses algılayan veya master'a ilk ulaşan node kazanan kabul edilir.
    winnerId = id;
}

seen[id] = true;

Serial.print("EVENT KAYDEDİLDİ | Node: ");
Serial.println(id);
}

void sendResult(uint8_t winner) {
    ResultPacket result;
    result.msgType = MSG_RESULT;
    result.winnerId = winner;
    result.decisionId = ++decisionId;

    for (int i = 0; i < 5; i++) {
        esp_now_send(broadcastAddress, (uint8_t *)&result, sizeof(result));
        delay(30);
    }

    Serial.println();
    Serial.println("===== KARAR =====");
    Serial.print("Sese en yakın / ilk algılayan Node: ");
    Serial.println(winner);

    for (int i = 1; i <= 3; i++) {
        Serial.print("Node ");
        Serial.print(i);
        Serial.print(" algıladı mı: ");
        Serial.println(seen[i] ? "EVET" : "HAYIR");
    }

    Serial.println("=====");
    Serial.println();

    if (winner == NODE_ID) {
        setRed();
    } else {
        setGreen();
    }

    resultSent = true;
    resultStartMs = millis();
}

```

```

}

void handleEventPacket(const uint8_t *incomingData, int len) {
    if (len != sizeof(EventPacket)) return;

    EventPacket pkt;
    memcpy(&pkt, incomingData, sizeof(pkt));

    if (pkt.msgType != MSG_EVENT) return;
    if (pkt.nodeId == NODE_ID) return;

    registerEvent(pkt.nodeId);
}

#if defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3
void OnDataRecv(const esp_now_recv_info_t *recv_info, const uint8_t
*incomingData, int len) {
#else
void OnDataRecv(const uint8_t *mac, const uint8_t *incomingData, int len) {
#endif
    if (len <= 0) return;

    uint8_t msgType = incomingData[0];

    if (msgType == MSG_EVENT) {
        handleEventPacket(incomingData, len);
    }
}

void resetRound() {
    roundActive = false;
    resultSent = false;
    winnerId = 0;

    for (int i = 0; i < 4; i++) {
        seen[i] = false;
    }

    setGreen();
}

void setup() {
    Serial.begin(115200);
    delay(1000);

    Serial.println();
    Serial.println("MASTER + NODE 1 DIJITAL MIKROFON BASLIYOR...");

    pinMode(MIC_OUT_PIN, INPUT);
}

```

```

pinMode(RED_LED_PIN, OUTPUT);
pinMode(GREEN_LED_PIN, OUTPUT);

setGreen();

WiFi.mode(WIFI_STA);
WiFi.disconnect();

esp_wifi_set_channel(ESPNOW_CHANNEL, WIFI_SECOND_CHAN_NONE);

Serial.print("MASTER / NODE 1 MAC: ");
Serial.println(WiFi.macAddress());

if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW BASLATILAMADI!");
    setRed();
    return;
}

esp_now_register_recv_cb(OnDataRecv);

esp_now_peer_info_t peerInfo = {};
memcpy(peerInfo.peer_addr, broadcastAddress, 6);
peerInfo.channel = ESPNOW_CHANNEL;
peerInfo.encrypt = false;

if (!esp_now_is_peer_exist(broadcastAddress)) {
    if (esp_now_add_peer(&peerInfo) != ESP_OK) {
        Serial.println("Broadcast peer eklenemedi!");
        setRed();
        return;
    }
}

resetRound();

Serial.println("MASTER HAZIR. Ses bekleniyor...");
}

void loop() {
    if (resultSent) {
        if (millis() - resultStartMs >= RESULT_HOLD_MS) {
            resetRound();
            Serial.println("Yeni ses olayi bekleniyor...");
        }
        return;
    }
}

int micState = digitalRead(MIC_OUT_PIN);

```

```

static uint32_t lastPrintMs = 0;
if (millis() - lastPrintMs > 300) {
    lastPrintMs = millis();
    Serial.print("Node 1 MIC OUT = ");
    Serial.println(micState);
}

if (micState == SOUND_ACTIVE_STATE &&
    millis() - lastLocalEventMs > LOCAL_COOLDOWN_MS) {

    lastLocalEventMs = millis();
    localEventId++;

    Serial.println("NODE 1 SES ALGILADI!");

    registerEvent(NODE_ID);
}

if (roundActive && !resultSent) {
    if (millis() - roundStartMs >= DECISION_WINDOW_MS) {
        if (winnerId != 0) {
            sendResult(winnerId);
        } else {
            resetRound();
        }
    }
}
}
}
}

```

## EK-B: Node 2 / Node 3 Ortak Mikrofonlu ESP-NOW Kodu

**Not:** Bu kod Node 2 ve Node 3 için ortaktır. Node 2'ye yüklerken:

```
#define NODE_ID 2
```

Node 3'e yüklerken:

```
#define NODE_ID 3
```

Yapılmalıdır.

```

#include <esp_now.h>
#include <WiFi.h>
#include <esp_wifi.h>

#if __has_include(<esp_arduino_version.h>)
    #include <esp_arduino_version.h>
#endif

// NODE 2 için 2 yap.
// NODE 3 için 3 yap.
#define NODE_ID 2

#define ESPNOW_CHANNEL 1

#define MIC_OUT_PIN 32

#define RED_LED_PIN 26
#define GREEN_LED_PIN 27

#define MSG_EVENT 1
#define MSG_RESULT 2

// Kullanılan mikrofon modülü ses algıladığında OUT pini LOW seviyesine
düşmektedir.
#define SOUND_ACTIVE_STATE LOW

#define SEND_REPEAT 3
#define SEND_COOLDOWN_MS 1200
#define RESULT_HOLD_MS 3000

uint8_t broadcastAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

typedef struct __attribute__((packed)) {
    uint8_t msgType;
    uint8_t nodeId;
    uint32_t eventId;
    uint32_t eventTime;
} EventPacket;

typedef struct __attribute__((packed)) {
    uint8_t msgType;
    uint8_t winnerId;
    uint32_t decisionId;
} ResultPacket;

bool resultMode = false;

uint32_t resultStartMs = 0;
uint32_t lastSendMs = 0;

```



```

uint32_t eventId = 0;

void setGreen() {
    digitalWrite(GREEN_LED_PIN, HIGH);
    digitalWrite(RED_LED_PIN, LOW);
}

void setRed() {
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, HIGH);
}

void alloff() {
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, LOW);
}

void sendEvent() {
    EventPacket pkt;
    pkt.msgType = MSG_EVENT;
    pkt.nodeId = NODE_ID;
    pkt.eventId = ++eventId;
    pkt.eventTime = millis();

    for (int i = 0; i < SEND_REPEAT; i++) {
        esp_now_send(broadcastAddress, (uint8_t *)&pkt, sizeof(pkt));
        delay(20);
    }

    Serial.print("EVENT GONDERILDI | Node: ");
    Serial.println(NODE_ID);
}

void handleResultPacket(const uint8_t *incomingData, int len) {
    if (len != sizeof(ResultPacket)) return;

    ResultPacket result;
    memcpy(&result, incomingData, sizeof(result));

    if (result.msgType != MSG_RESULT) return;
    if (result.winnerId < 1 || result.winnerId > 3) return;

    Serial.print("RESULT ALINDI | Kazanan Node: ");
    Serial.println(result.winnerId);

    resultMode = true;
    resultStartMs = millis();

    if (result.winnerId == NODE_ID) {

```

```

        setRed();
    } else {
        setGreen();
    }
}

#if defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3
void OnDataRecv(const esp_now_recv_info_t *recv_info, const uint8_t
*incomingData, int len) {
#else
void OnDataRecv(const uint8_t *mac, const uint8_t *incomingData, int len) {
#endif
    if (len <= 0) return;

    uint8_t msgType = incomingData[0];

    if (msgType == MSG_RESULT) {
        handleResultPacket(incomingData, len);
    }
}

void setup() {
    Serial.begin(115200);
    delay(1000);

    Serial.println();
    Serial.print("NODE ");
    Serial.print(NODE_ID);
    Serial.println(" BASLIYOR...");

    pinMode(MIC_OUT_PIN, INPUT);

    pinMode(RED_LED_PIN, OUTPUT);
    pinMode(GREEN_LED_PIN, OUTPUT);

    setGreen();

    WiFi.mode(WIFI_STA);
    WiFi.disconnect();

    esp_wifi_set_channel(ESPNOW_CHANNEL, WIFI_SECOND_CHAN_NONE);

    Serial.print("NODE MAC: ");
    Serial.println(WiFi.macAddress());

    if (esp_now_init() != ESP_OK) {
        Serial.println("ESP-NOW BASLATILAMADI!");
        setRed();
        return;
    }
}

```

```

}

esp_now_register_recv_cb(OnDataRecv);

esp_now_peer_info_t peerInfo = {};
memcpy(peerInfo.peer_addr, broadcastAddress, 6);
peerInfo.channel = ESPNOW_CHANNEL;
peerInfo.encrypt = false;

if (!esp_now_is_peer_exist(broadcastAddress)) {
    if (esp_now_add_peer(&peerInfo) != ESP_OK) {
        Serial.println("Broadcast peer eklenemedi!");
        setRed();
        return;
    }
}

Serial.print("NODE ");
Serial.print(NODE_ID);
Serial.println(" HAZIR. Ses bekleniyor...");
}

void loop() {
    if (resultMode) {
        if (millis() - resultStartMs >= RESULT_HOLD_MS) {
            resultMode = false;
            setGreen();

            Serial.print("NODE ");
            Serial.print(NODE_ID);
            Serial.println(" tekrar dinleme modunda.");
        }
        return;
    }

    int micState = digitalRead(MIC_OUT_PIN);

    static uint32_t lastPrintMs = 0;
    if (millis() - lastPrintMs > 300) {
        lastPrintMs = millis();

        Serial.print("Node ");
        Serial.print(NODE_ID);
        Serial.print(" MIC OUT = ");
        Serial.println(micState);
    }

    if (micState == SOUND_ACTIVE_STATE &&
        millis() - lastSendMs > SEND_COOLDOWN_MS) {

```

```

    lastSendMs = millis();

    Serial.print("NODE ");
    Serial.print(NODE_ID);
    Serial.println(" SES ALGILADI!");

    sendEvent();
}
}

```

## EK-C: Mikrofon Modülü Test Kodu

Bu kod, mikrofon modülünün dijital OUT çıkışının nasıl çalıştığını test etmek için kullanılmıştır. Test sonucunda ses algılandığında OUT pininin LOW olduğu görülmüştür.

```

const int MIC_PIN = 32;

void setup() {
    Serial.begin(115200);
    pinMode(MIC_PIN, INPUT);

    Serial.println("Ses sensoru baslatildi. Dinleniyor...");
}

void loop() {
    int sesDurumu = digitalRead(MIC_PIN);

    Serial.print("OUT durumu: ");
    Serial.println(sesDurumu);

    if (sesDurumu == LOW) {
        Serial.println("Ses Algilandi! (El cirpmasi / Gurultu)");
        delay(200);
    }

    delay(50);
}

```

## EK-D: HC-SR04 Mesafe Sensörlü İlk MVP Kodu

Bu kod, projenin ilk MVP aşamasında kullanılmıştır. Master ESP32, HC-SR04 sensörüyle mesafe ölçmekte ve engel durumuna göre Node 2 ile Node 3'e ESP-NOW üzerinden komut göndermektedir.

- **EK-D.1: HC-SR04 Master Kodu**

```

#include <esp_now.h>
#include <WiFi.h>
#include <esp_wifi.h>

#define NODE_ID 1
#define ESPNOW_CHANNEL 1

#define TRIG_PIN 26
#define ECHO_PIN 27

#define OBSTACLE_ON_CM 20.0
#define OBSTACLE_OFF_CM 25.0

#define MIN_VALID_CM 2.0
#define MAX_VALID_CM 400.0

#define MSG_COMMAND 1
#define SEND_INTERVAL_MS 300

uint8_t broadcastAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

typedef struct __attribute__((packed)) {
    uint8_t msgType;
    uint8_t obstacle;      // 0 = engel yok, 1 = engel var
    float distanceCm;
    uint32_t commandId;
} CommandPacket;

uint32_t lastSendMs = 0;
uint32_t commandId = 0;

bool obstacleState = false;

float readRawDistanceCm() {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(3);

    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);

    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000);

    if (duration == 0) {
        return -1.0;
    }

    float distance = duration * 0.0343 / 2.0;

```

```

    return distance;
}

float readFilteredDistanceCm() {
    const int sampleCount = 5;
    float validSum = 0;
    int validCount = 0;

    for (int i = 0; i < sampleCount; i++) {
        float d = readRawDistanceCm();

        if (d >= MIN_VALID_CM && d <= MAX_VALID_CM) {
            validSum += d;
            validCount++;
        }

        delay(30);
    }

    if (validCount == 0) {
        return -1.0;
    }

    return validSum / validCount;
}

void updateObstacleState(float distanceCm) {
    if (distanceCm < 0) {
        return;
    }

    if (distanceCm <= OBSTACLE_ON_CM) {
        obstacleState = true;
    }
    else if (distanceCm >= OBSTACLE_OFF_CM) {
        obstacleState = false;
    }
}

void sendCommand(bool obstacle, float distanceCm) {
    CommandPacket cmd;
    cmd.msgType = MSG_COMMAND;
    cmd.obstacle = obstacle ? 1 : 0;
    cmd.distanceCm = distanceCm;
    cmd.commandId = ++commandId;

    esp_now_send(broadcastAddress, (uint8_t *)&cmd, sizeof(cmd));

    Serial.print("MESAFE: ");

```

```

Serial.print(distanceCm);
Serial.print(" cm | DURUM: ");

if (obstacle) {
    Serial.println("ENGEL VAR -> KIRMIZI");
} else {
    Serial.println("ENGEL YOK -> YESIL");
}
}

void setup() {
    Serial.begin(115200);
    delay(1000);

    Serial.println();
    Serial.println("MASTER + HC-SR04 BASLIYOR...");

    pinMode(TRIG_PIN, OUTPUT);
    pinMode(ECHO_PIN, INPUT);

    digitalWrite(TRIG_PIN, LOW);

    WiFi.mode(WIFI_STA);
    WiFi.disconnect();

    esp_wifi_set_channel(ESPNOW_CHANNEL, WIFI_SECOND_CHAN_NONE);

    Serial.print("MASTER MAC: ");
    Serial.println(WiFi.macAddress());

    if (esp_now_init() != ESP_OK) {
        Serial.println("ESP-NOW BASLATILAMADI!");
        return;
    }

    esp_now_peer_info_t peerInfo = {};
    memcpy(peerInfo.peer_addr, broadcastAddress, 6);
    peerInfo.channel = ESPNOW_CHANNEL;
    peerInfo.encrypt = false;

    if (!esp_now_is_peer_exist(broadcastAddress)) {
        if (esp_now_add_peer(&peerInfo) != ESP_OK) {
            Serial.println("Broadcast peer eklenemedi!");
            return;
        }
    }

    Serial.println("MASTER HAZIR.");
}

```

```

void loop() {
  if (millis() - lastSendMs >= SEND_INTERVAL_MS) {
    lastSendMs = millis();

    float distance = readFilteredDistanceCm();

    if (distance < 0) {
      Serial.println("Gecerli mesafe okunamadi. Onceki durum korunuyor.");
    }

    updateObstacleState(distance);
    sendCommand(obstacleState, distance);
  }
}

```

- **EK-D.2: HC-SR04 Node 2 / Node 3 LED Alıcı Kodu**

Node 2 ve Node 3 için aynı kod kullanılır. Bu sistemde node'lar sadece master'dan gelen engel komutuna göre LED yakar.

```

#include <esp_now.h>
#include <WiFi.h>
#include <esp_wifi.h>

#if __has_include(<esp_arduino_version.h>)
  #include <esp_arduino_version.h>
#endif

#define NODE_ID 2
#define ESPNOW_CHANNEL 1

#define GREEN_LED_PIN 27
#define RED_LED_PIN 26

#define MSG_COMMAND 1

typedef struct __attribute__((packed)) {
  uint8_t msgType;
  uint8_t obstacle;
  float distanceCm;
  uint32_t commandId;
} CommandPacket;

void setGreen() {
  digitalWrite(GREEN_LED_PIN, HIGH);
  digitalWrite(RED_LED_PIN, LOW);
}

```



```

void setRed() {
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, HIGH);
}

void alloff() {
    digitalWrite(GREEN_LED_PIN, LOW);
    digitalWrite(RED_LED_PIN, LOW);
}

#if defined(ESP_ARDUINO_VERSION_MAJOR) && ESP_ARDUINO_VERSION_MAJOR >= 3
void OnDataRecv(const esp_now_recv_info_t *recv_info, const uint8_t
*incomingData, int len) {
#else
void OnDataRecv(const uint8_t *mac, const uint8_t *incomingData, int len) {
#endif

    if (len != sizeof(CommandPacket)) return;

    CommandPacket cmd;
    memcpy(&cmd, incomingData, sizeof(cmd));

    if (cmd.msgType != MSG_COMMAND) return;

    if (cmd.obstacle == 1) {
        setRed();
    } else {
        setGreen();
    }

    Serial.print("NODE ");
    Serial.print(NODE_ID);
    Serial.print(" KOMUT ALDI | Mesafe: ");
    Serial.print(cmd.distanceCm);
    Serial.print(" cm | Durum: ");

    if (cmd.obstacle == 1) {
        Serial.println("ENGEL VAR -> KIRMIZI");
    } else {
        Serial.println("ENGEL YOK -> YESIL");
    }
}

void setup() {
    Serial.begin(115200);
    delay(1000);

    Serial.println();

```

```

Serial.print("NODE ");
Serial.print(NODE_ID);
Serial.println(" BASLIYOR...");

pinMode(GREEN_LED_PIN, OUTPUT);
pinMode(RED_LED_PIN, OUTPUT);

allOff();

WiFi.mode(WIFI_STA);
WiFi.disconnect();

esp_wifi_set_channel(ESPNOW_CHANNEL, WIFI_SECOND_CHAN_NONE);

Serial.print("NODE MAC: ");
Serial.println(WiFi.macAddress());

if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW BASLATILAMADI!");
    return;
}

esp_now_register_recv_cb(OnDataRecv);

Serial.println("NODE HAZIR. Master komutu bekleniyor...");
}

void loop() {
    // Node sadece master'dan gelen ESP-NOW komutunu bekler.}

```